

Incremental String Comparison

Gad M. Landau, Eugene W. Myers, Jeanette P. Schmidt

SIAM J. Comput., Vol. 27, No. 2, pp. 557-582 (1998)

Presenter: Tzu-Chiang Hsu (徐子強)

2015/03/31

Abstract (1/4)

The problem of comparing two sequences A and B to determine their longest common subsequence (LCS) or the edit distance between them has been much studied. In this paper we consider the following incremental version of these problems:

Abstract (2/4)

given an appropriate encoding of a comparison between A and B , can one incrementally compute the answer for A and bB , and the answer for A and Bb with equal efficiency, where b is an additional symbol?

Abstract (3/4)

Our main result is a theorem exposing a surprising relationship between the dynamic programming solutions for two such “adjacent” problems. Given a threshold k on the number of differences to be permitted in an alignment, the theorem leads directly to an $O(k)$ algorithm for incrementally computing a new solution from an old one, as contrasts the $O(k^2)$ time required to compute a solution from scratch.

Abstract (4/4)

We further show, with a series of applications, that this algorithm is indeed more powerful than its nonincremental counterpart. We show this by solving the applications with greater asymptotic efficiency than heretofore possible. For example, we obtain $O(nk)$ algorithms for the longest prefix approximate match problem, the approximate overlap problem, and cyclic string comparison.

		c	t	g	a	g	c	t	g	a	a
t	0	1	2	3	4	5	6	7	8	9	10
a	1	1	1	2	3	4	5	6	7	8	9
g	2	2	2	2	2	3	4	5	6	7	8
c	3	3	3	2	3	2	3	4	5	6	7
c	4	3	4	3	3	3	2	3	4	5	6
t	5	4	4	4	4	4	3	3	4	5	6
a	6	5	4	5	5	5	4	3	4	5	6
g	7	6	5	5	5	6	5	4	4	4	5
a	8	7	6	5	6	5	6	5	4	5	5
t	9	8	7	6	5	6	6	6	5	4	5
c	10	9	8	7	6	6	7	6	6	5	5
	11	10	9	8	7	7	6	7	7	6	6

indel: 1
substitution: 1

0

		c	t	g	a	g	c	t	g	a	a
	0	1	2	3	4	5	6	7	8	9	10
t	1	1	1	2	3	4	5	6	7	8	9
a	2	2	2	2	2	3	4	5	6	7	8
g	3	3	3	2	3	2	3	4	5	6	7
c	4	3	4	3	3	3	2	3	4	5	6
c	5	4	4	4	4	4	3	3	4	5	6
t	6	5	4	5	5	5	4	3	4	5	6
a	7	6	5	5	5	6	5	4	4	4	5
g	8	7	6	5	6	5	6	5	4	5	5
a	9	8	7	6	5	6	6	6	5	4	5
t	10	9	8	7	6	6	7	6	6	5	5
c	11	10	9	8	7	7	6	7	7	6	6

1

		c	t	g	a	g	c	t	g	a	a
	0	1	2	3	4	5	6	7	8	9	10
t	1	1	1	2	3	4	5	6	7	8	9
a	2	2	2	2	2	3	4	5	6	7	8
g	3	3	3	3	2	3	2	3	4	5	6
c	4	3	4	3	3	3	2	3	4	5	6
c	5	4	4	4	4	4	3	3	4	5	6
t	6	5	4	5	5	5	4	3	4	5	6
a	7	6	5	5	5	6	5	4	4	4	5
g	8	7	6	5	6	5	6	5	4	5	5
a	9	8	7	6	5	6	6	6	5	4	5
t	10	9	8	7	6	6	7	6	6	5	5
c	11	10	9	8	7	7	6	7	7	6	6

0		c	t	g	a	g	c	t	g	a	a
	0	1	2	3	4	5	6	7	8	9	10
t	1	1	1	2	3	4	5	6	7	8	9
a	2	2	2	2	2	3	4	5	6	7	8
g	3	3	3	2	3	2	3	4	5	6	7
c	4	3	4	3	3	3	2	3	4	5	6
c	5	4	4	4	4	4	3	3	4	5	6
t	6	5	4	5	5	5	4	3	4	5	6
a	7	6	5	5	5	6	5	4	4	4	5
g	8	7	6	5	6	5	6	5	4	5	5
a	9	8	7	6	5	6	6	6	5	4	5
t	10	9	8	7	6	6	7	6	6	5	5
c	11	10	9	8	7	7	6	7	7	6	6

	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:							0					
1:						1	1	1				
2:					2	2	3	2	4			
3:				4	3	4	4	6	5	4		
4:			6	5	5	5	9	7	7	5	4	
5:		9	7	8	6	10	10	9	8	7	5	4

tagcatc

[illegible][illegible]

0		c	t	g	a	g	c	t	g	a	a
	0	1	2	3	4	5	6	7	8	9	10
t	1	1	1	2	3	4	5	6	7	8	9
a	2	2	2	2	2	3	4	5	6	7	8
g	3	3	3	2	3	2	3	4	5	6	7
c	4	3	4	3	3	3	2	3	4	5	6
c	5	4	4	4	4	4	3	3	4	5	6
t	6	5	4	5	5	5	4	3	4	5	6
a	7	6	5	5	5	6	5	4	4	4	5
g	8	7	6	5	6	5	6	5	4	5	5
a	9	8	7	6	5	6	6	6	5	4	5
t	10	9	8	7	6	6	7	6	6	5	5
c	11	10	9	8	7	7	6	7	7	6	6

	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:							0					
1:						1	1	1				
2:					2	2	3	2	4			
3:				4	3	4	4	6	5	4		
4:			6	5	5	5	9	7	7	5	4	
5:		9	7	8	6	10	10	9	8	7	5	4

$$O(n + 3 \sum_{m=0}^k (2m+1))$$

$$= O(n + k^2)$$

0		c	t	g	a	g	c	t	g	a	a
	0	1	2	3	4	5	6	7	8	9	10
t	1	1	1	2	3	4	5	6	7	8	9
a	2	2	2	2	2	3	4	5	6	7	8
g	3	3	3	2	3	2	3	4	5	6	7
c	4	3	4	3	3	3	2	3	4	5	6
c	5	4	4	4	4	4	3	3	4	5	6
t	6	5	4	5	5	5	4	3	4	5	6
a	7	6	5	5	5	6	5	4	4	5	5
g	8	7	6	5	6	5	6	5	4	5	5
a	9	8	7	6	5	6	6	6	5	4	5
t	10	9	8	7	6	7	6	6	6	5	5
c	11	10	9	8	7	7	6	7	7	6	6

old:

	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:							0					
1:						1	1	1				
2:					2	2	3	2	4			
3:				4	3	4	4	6	5	4		
4:			6	5	5	5	9	7	7	5	4	
5:		9	7	8	6	10	10	9	8	7	5	4

old_4

-1

t

c t g a g c t g a a

	0	1	2	3	4	5	6	7	8	9	10	11
t	1	0	1	2	3	4	5	6	7	8	9	10
a	2	1	1	2	3	3	4	5	6	7	8	9
g	3	2	2	2	2	3	3	4	5	6	7	8
c	4	3	2	3	3	3	4	3	4	5	6	7
c	5	4	3	3	4	4	4	4	4	5	6	7
t	6	5	4	3	4	5	5	5	4	5	6	7
a	7	6	5	4	4	4	5	6	5	5	5	6
g	8	7	6	5	4	5	4	5	6	5	6	6
a	9	8	7	6	5	4	5	5	6	6	5	6
t	10	9	8	7	6	5	5	6	5	6	6	6
c	11	10	9	8	7	6	6	5	6	6	7	7

new:

	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:						1						
1:					2	2	1					
2:				4	3	3	3	1				
3:			6	5	4	4	4	3	4			
4:		9	7	8	5	5	5	6	5	4		
5:	10	11	9	10	8	6	9	7	7	5	4	

old:

	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:							0					
1:						1	1	1				
2:					2	2	3	2	4			
3:				4	3	4	4	6	5	4		
4:			6	5	5	5	9	7	7	5	4	
5:		9	7	8	6	10	10	9	8	7	5	4

Construct **new_i** from **new_{i-1}**, **old_{i-1}**,
old_i and **old_{i+1}** in $O(1)$ time.

new:

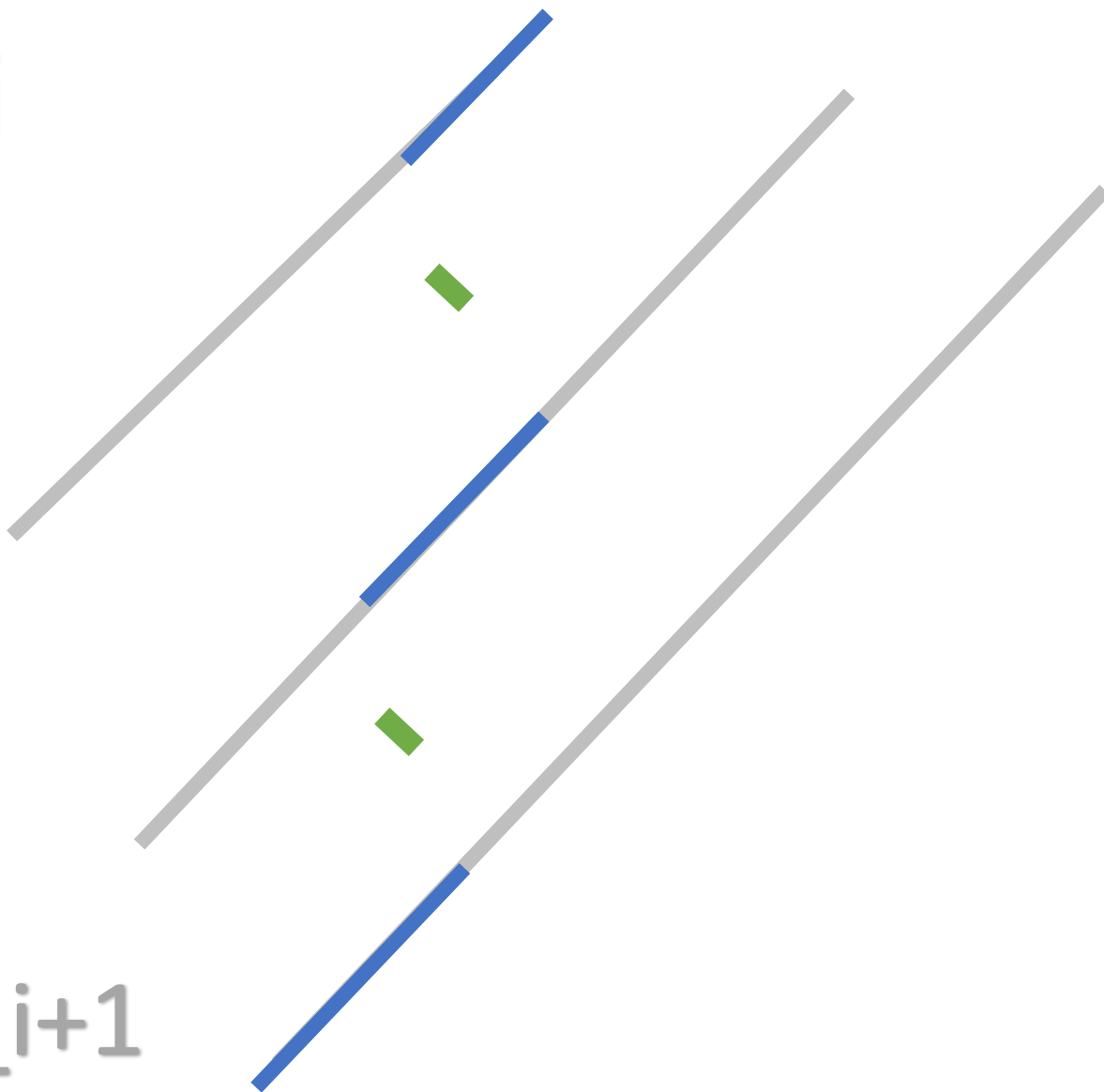
	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5
0:						1						
1:					2	2	1					
2:				4	3	3	3	1				
3:			6	5	4	4	4	3	4			
4:		9	7	8	5	5	5	6	5	4		
5:	10	11	9	10	8	6	9	7	7	5	4	

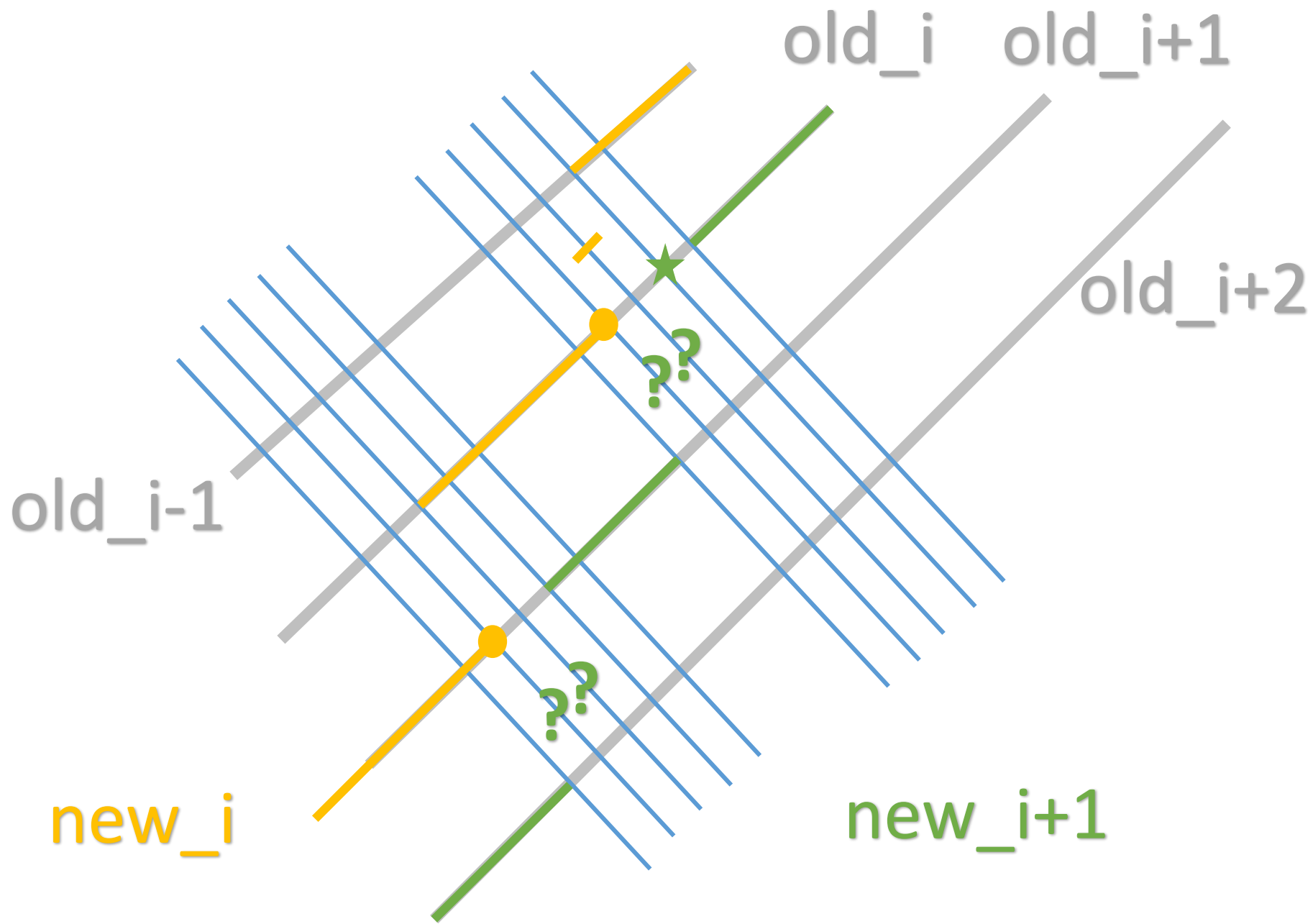
new_i

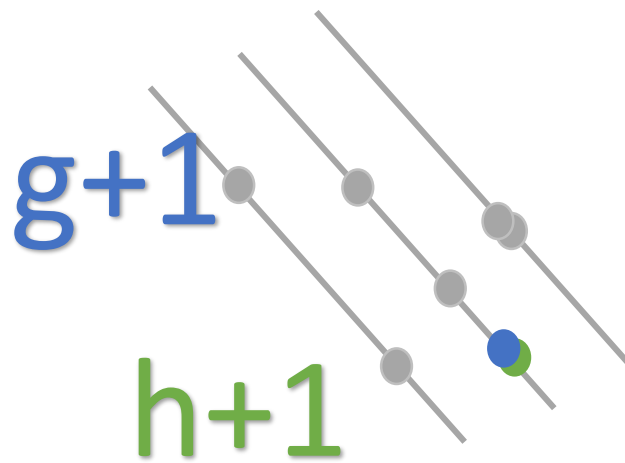
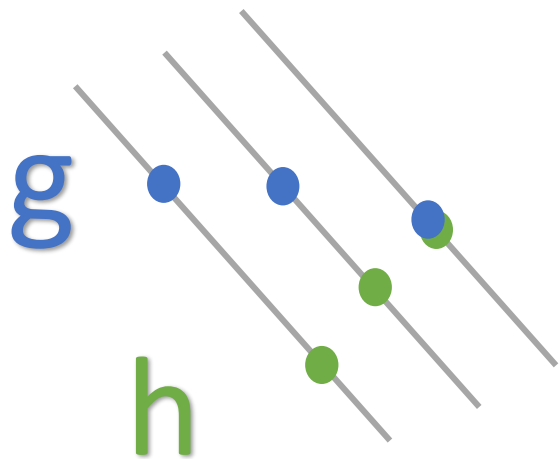
old_i-1

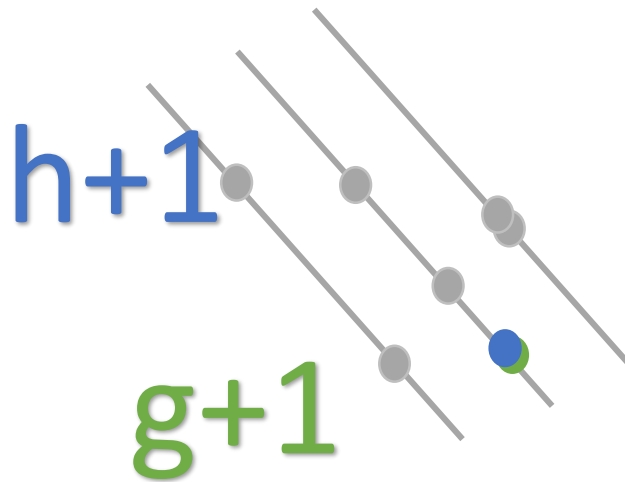
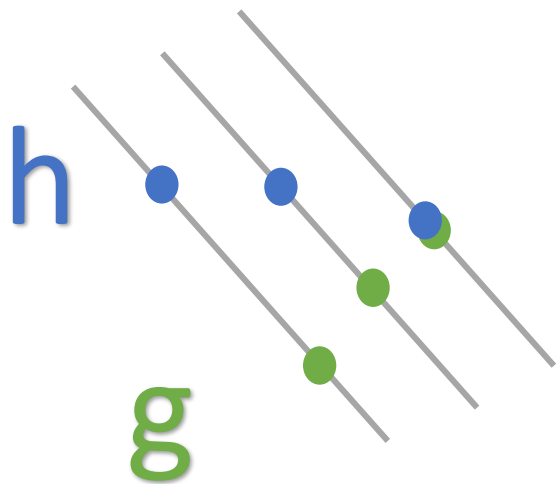
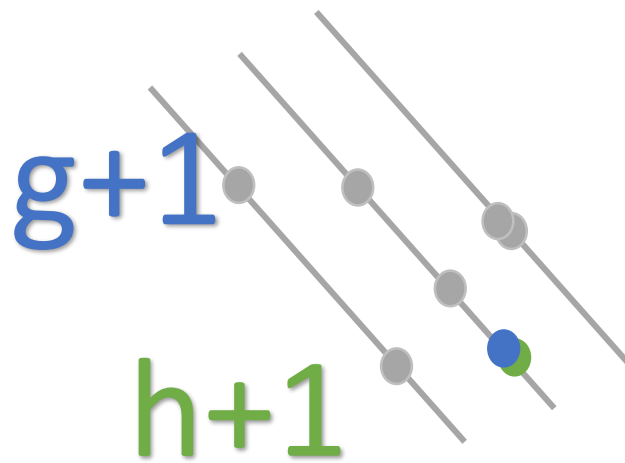
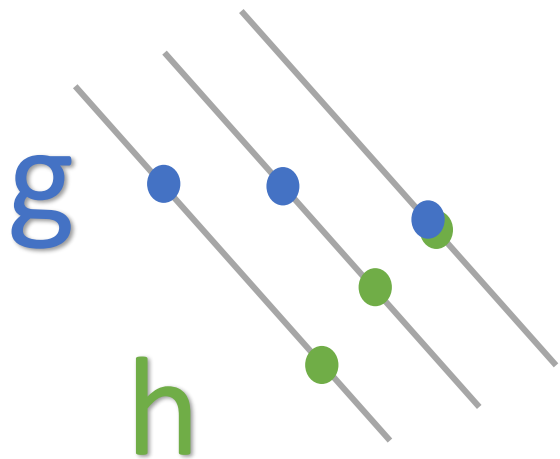
old_i

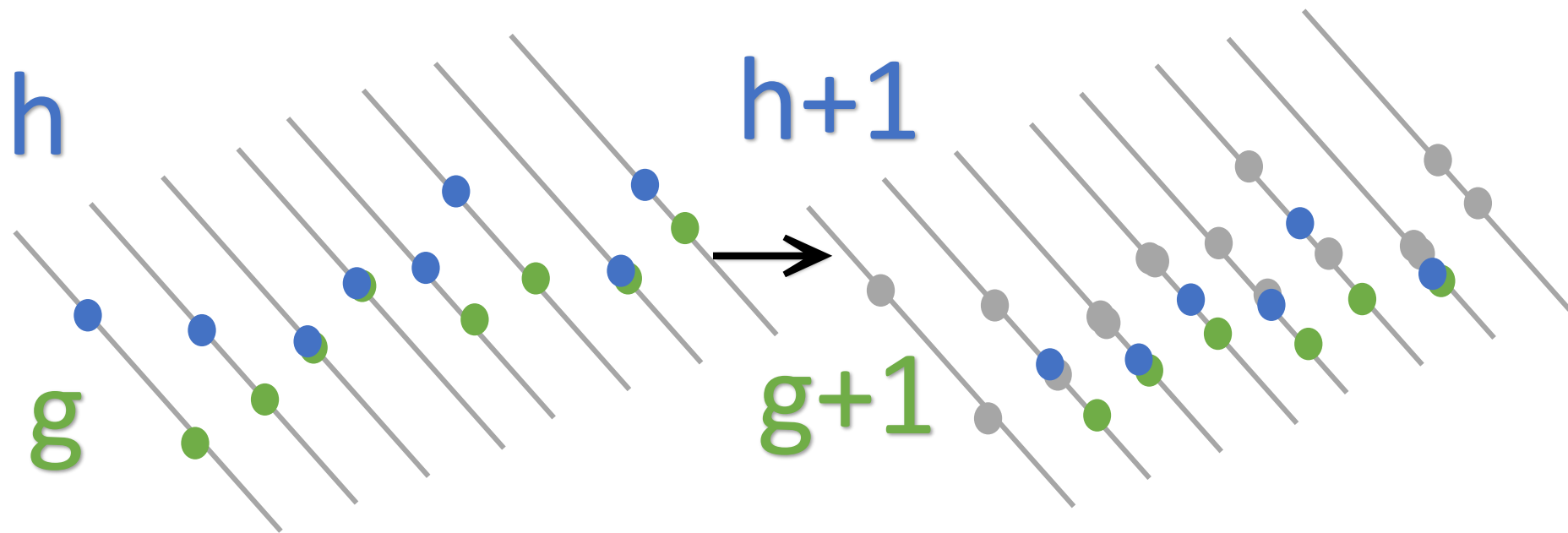
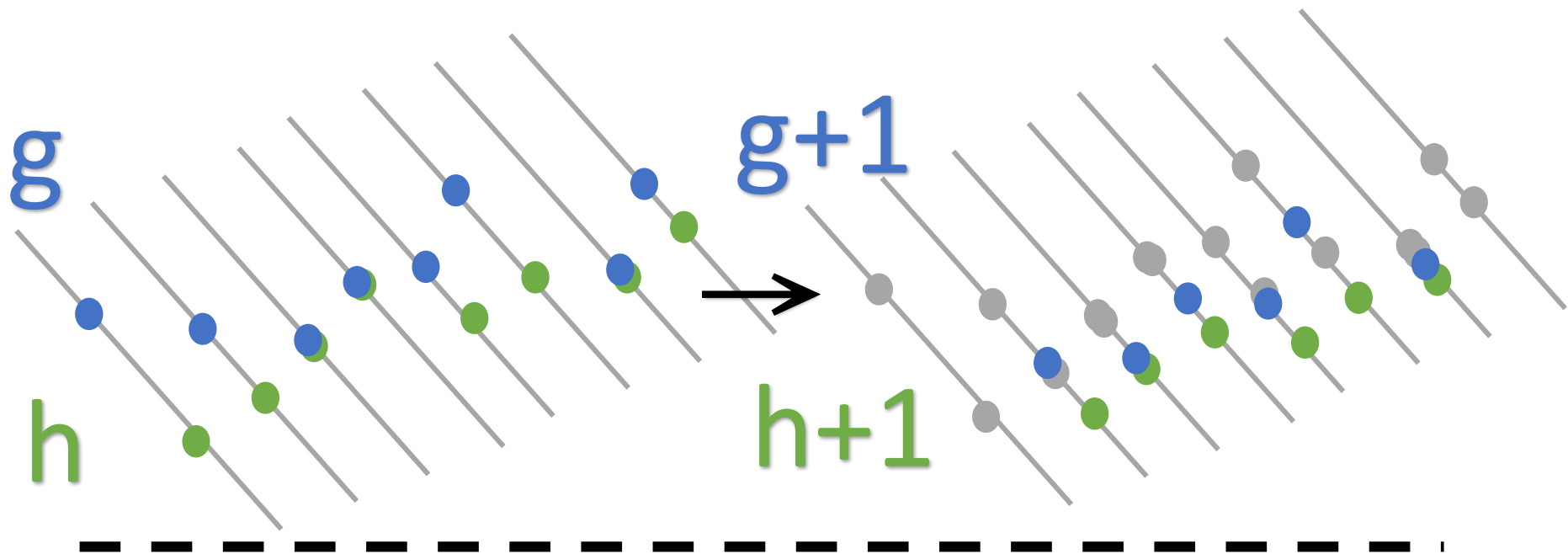
old_i+1

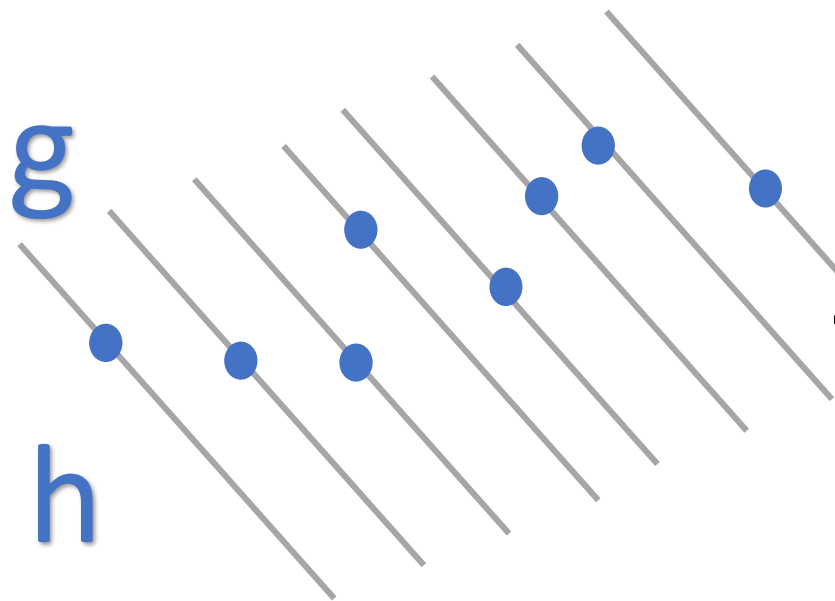








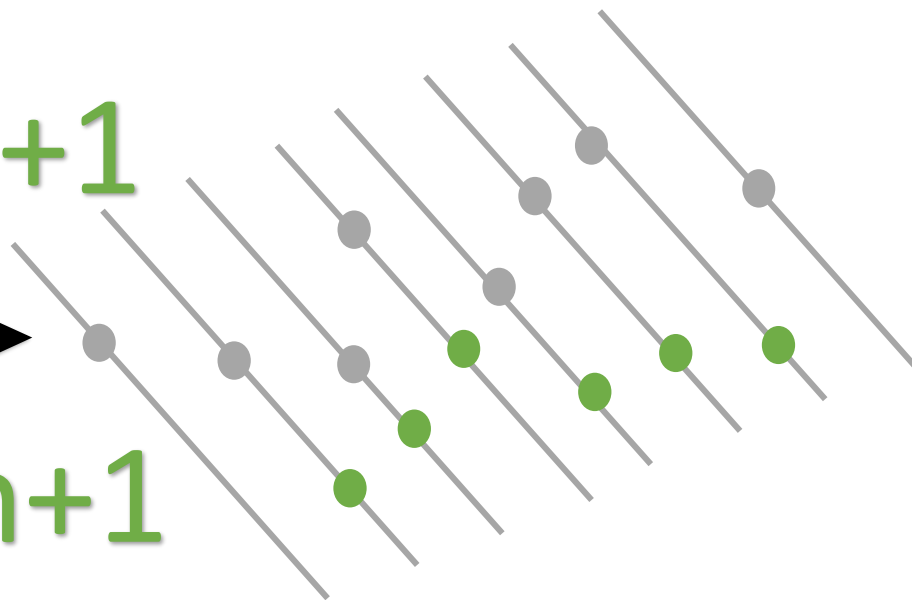


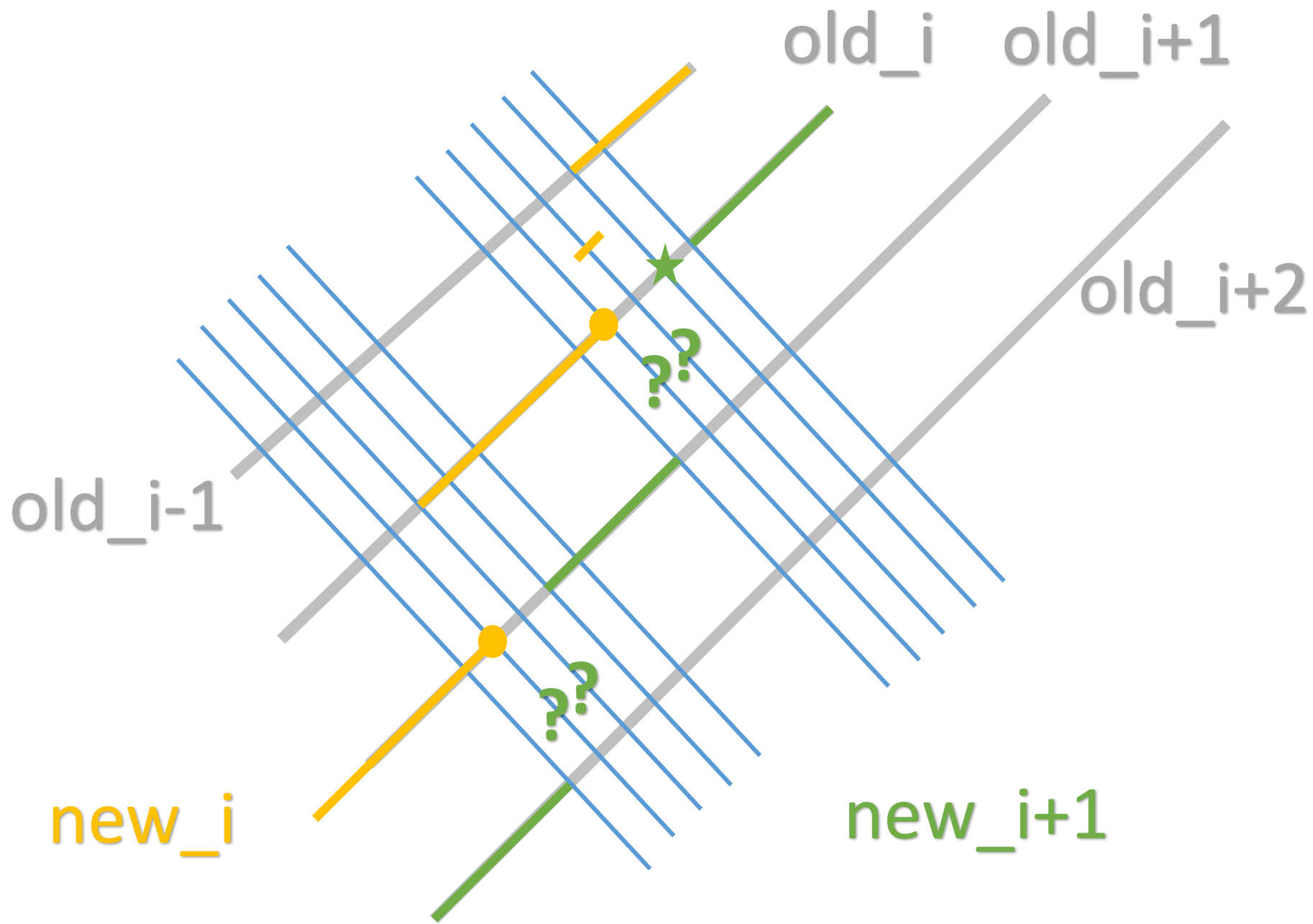


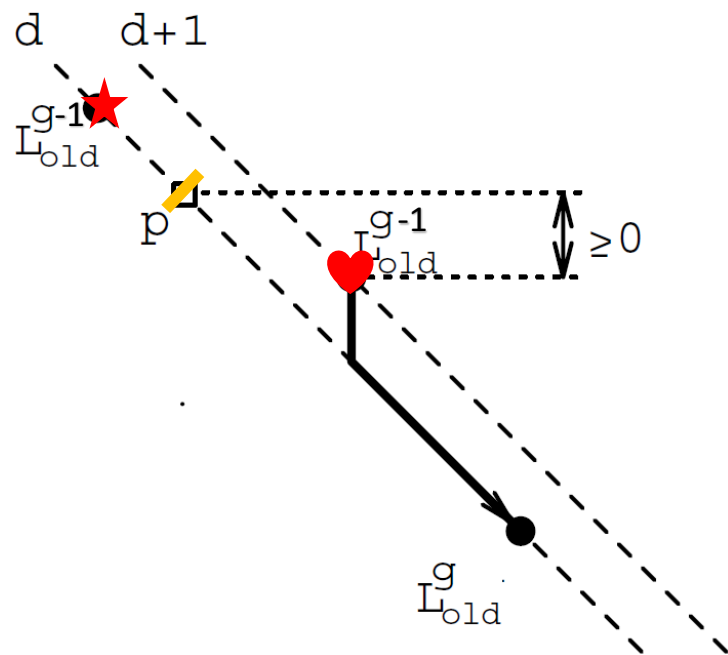
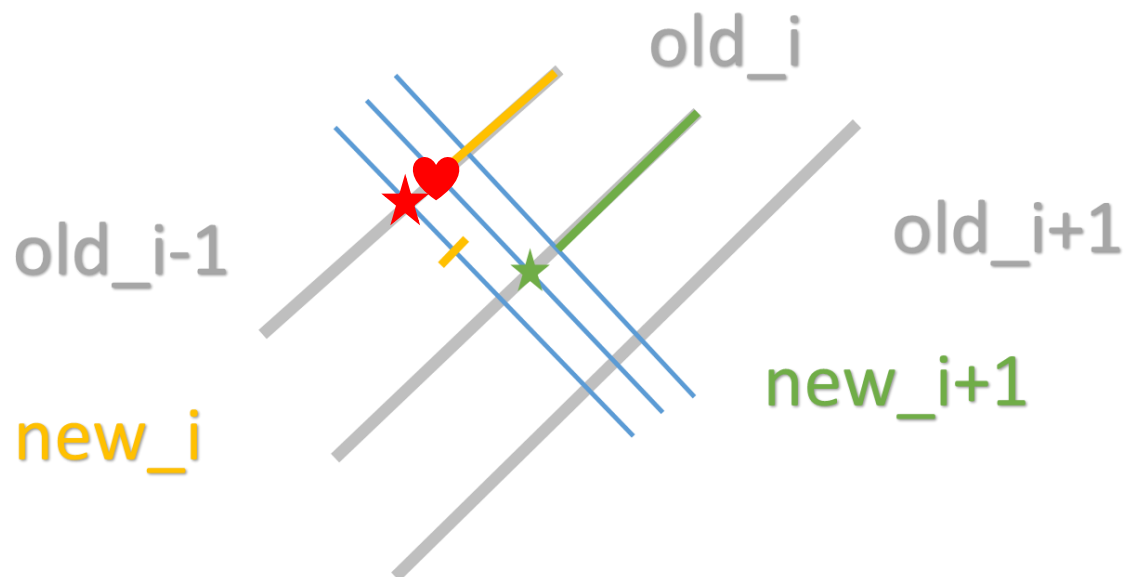
$g+1$

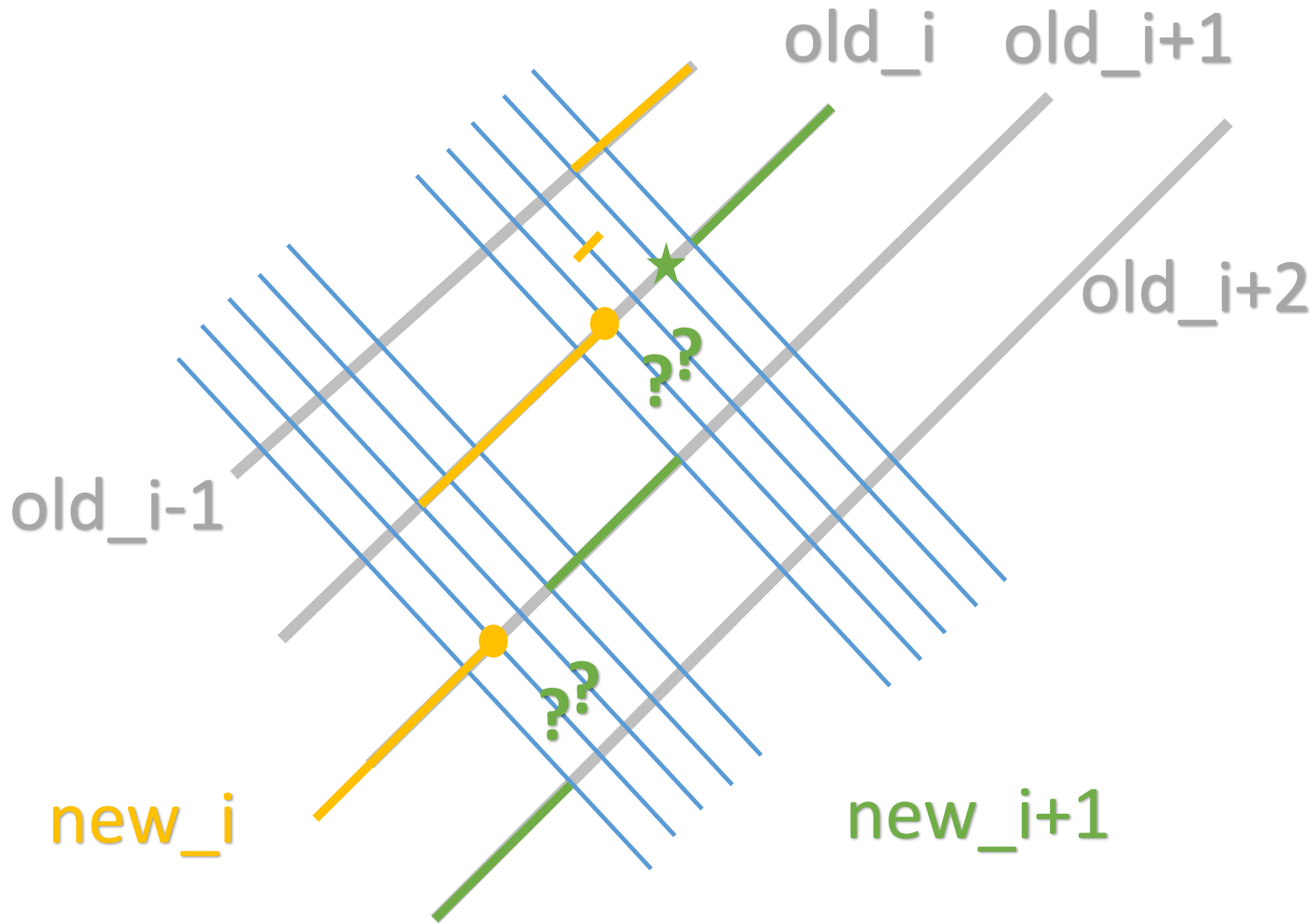


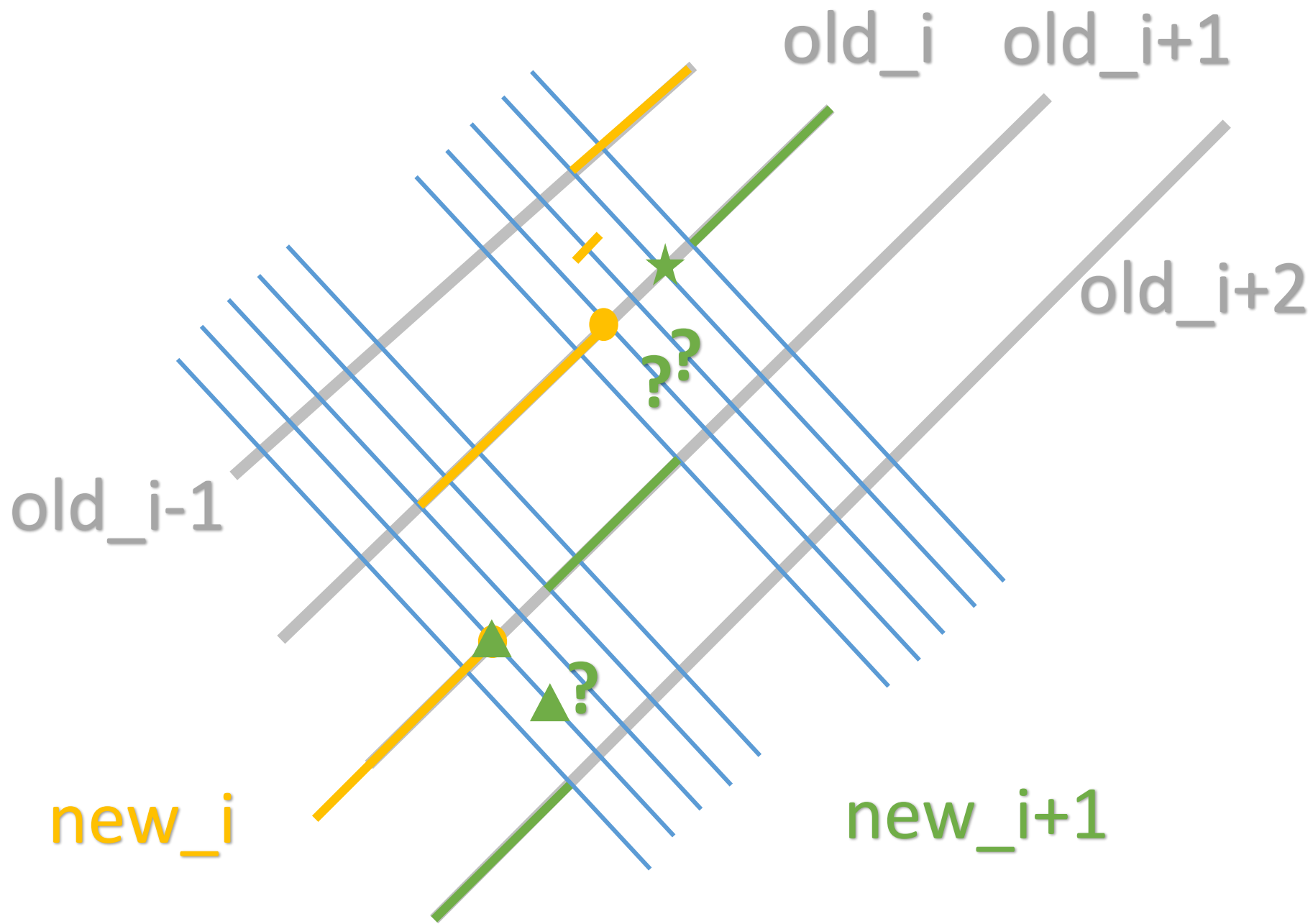
$h+1$

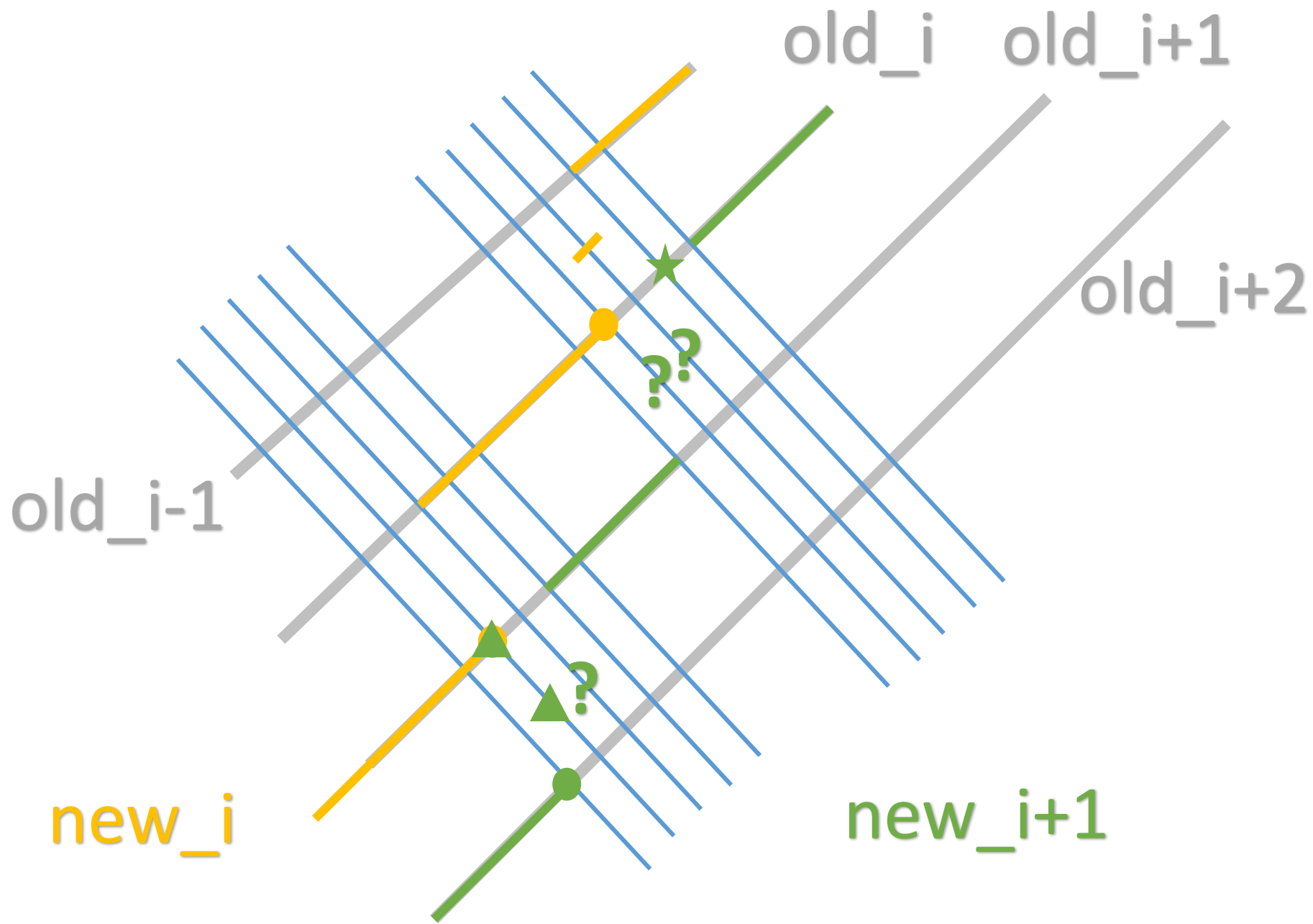


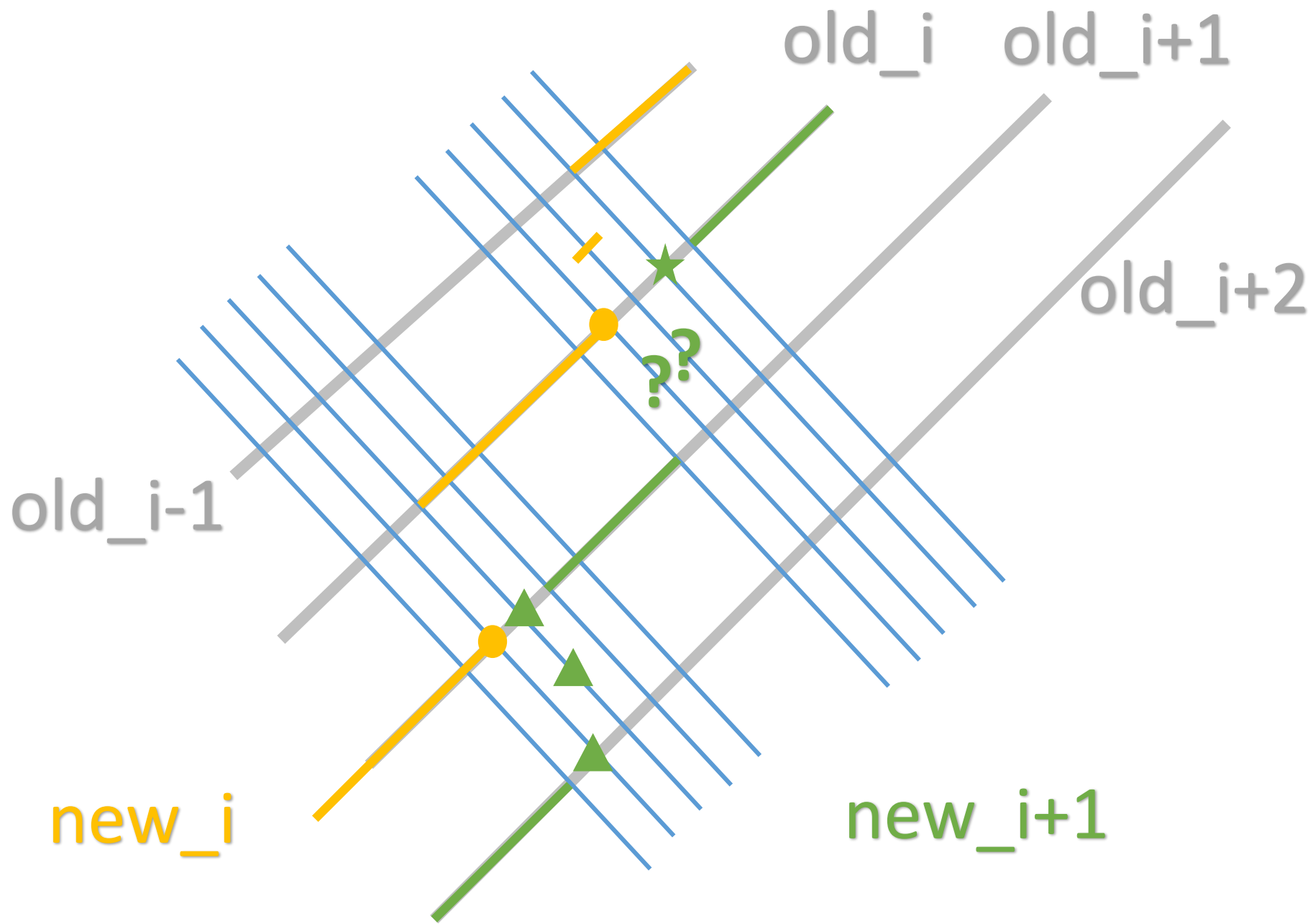


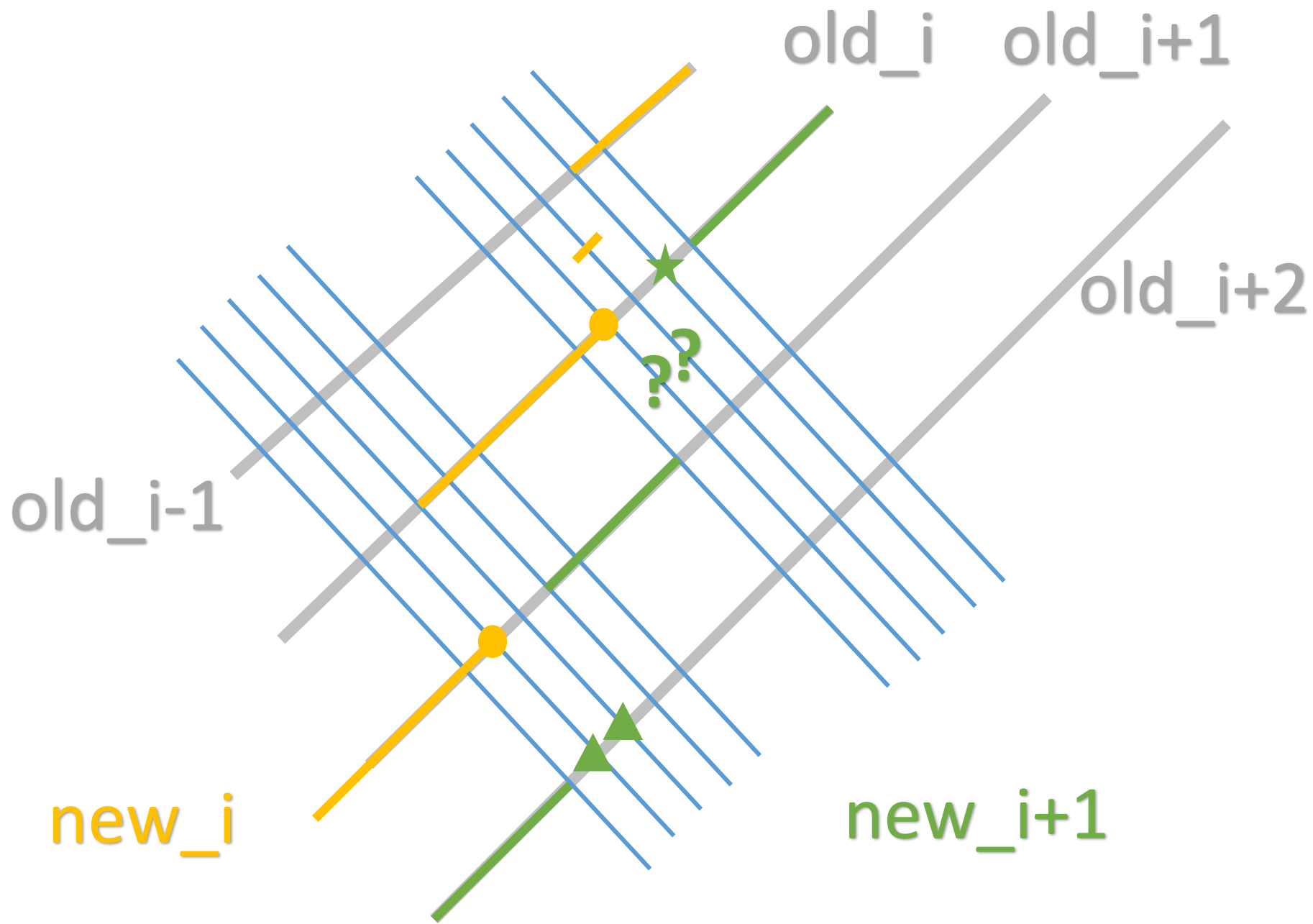












Procedure Construct_new_wave($h, p_1^h, p_2^h, p_1^{h+1}, p_2^{h+1}$)

1. $L_{new}^{h+1}[-h-2 \dots p_1^h-1] \leftarrow L_{old}^{h+2}[-h-2 \dots p_1^h-1]$
2. Compute($L_{new}^{h+1}(p_1^h)$)
3. Compute($L_{new}^{h+1}(p_1^h + 1)$)
4. $p_1^{h+1} \leftarrow \begin{cases} p_1^h - 1 & \text{if } L_{new}^{h+1}(p_1^h) < L_{old}^{h+2}(p_1^h) \\ p_1^h & \text{if } L_{new}^{h+1}(p_1^h) = L_{old}^{h+2}(p_1^h) \text{ and } L_{new}^{h+1}(p_1^h + 1) < L_{old}^{h+2}(p_1^h + 1) \\ p_1^h + 1 & \text{otherwise} \end{cases}$
5. **if** $p_2^h > p_1^h + 2$ **then**
6. $\{ L_{new}^{h+1}[p_1^h+2 \dots p_2^h-1] \leftarrow L_{old}^{h+1}[p_1^h+2 \dots p_2^h-1]$
7. Double_Link($L_{new}^{h+1}(p_1^h + 2), L_{new}^{h+1}(p_1^h + 1)$)
8. $\}$
9. **if** $p_2^h \geq p_1^h + 2$ **then** Compute($L_{new}^{h+1}(p_2^h)$)
10. **if** $p_2^h \geq p_1^h + 1$ **then** Compute($L_{new}^{h+1}(p_2^h + 1)$)
11. $p_2^{h+1} \leftarrow \begin{cases} p_2^h - 1 & \text{if } L_{new}^{h+1}(p_2^h) < L_{old}^{h+1}(p_2^h) \\ p_2^h & \text{if } L_{new}^{h+1}(p_2^h) = L_{old}^{h+1}(p_2^h) \text{ and } L_{new}^{h+1}(p_2^h + 1) < L_{old}^{h+1}(p_2^h + 1) \\ p_2^h + 1 & \text{otherwise} \end{cases}$
12. **if** $p_2^h + 2 \leq h$ **then**
13. $\{ L_{new}^{h+1}[p_2^h+2 \dots h] \leftarrow L_{old}^h[p_2^h+2 \dots h]$
14. Double_Link($L_{new}^{h+1}(p_2^h + 2), L_{new}^{h+1}(p_2^h + 1)$)
15. $\}$

FIG. 5. Construction of L_{new}^{h+1} from L_{new}^h and auxiliary pointers.

Procedure Compute($L_x^h(d)$)

1. **if** $x = new$ **then** $\delta \leftarrow 1$ **else** $\delta \leftarrow 0$
2. $L_x^h(d) \leftarrow Slide_d \left(\max \left\{ \begin{array}{ll} L_x^{h-1}(d+1) + 1 & \text{if } d < h - 1 - \delta \\ L_x^{h-1}(d) + 1 & \text{if } -h - \delta < d < h - \delta \\ L_x^{h-1}(d-1) & \text{if } d > -h + 1 - \delta \end{array} \right\} \right)$
3. **if** $h > 0$ **and** $d > -h - \delta$ **then**
4. { Double_Link($L_x^h(d)$, $L_x^h(d-1)$)
5. **if** $d < h - \delta$ **then** Double_Link($L_x^h(d)$, $L_x^{h-1}(d)$)
6. }

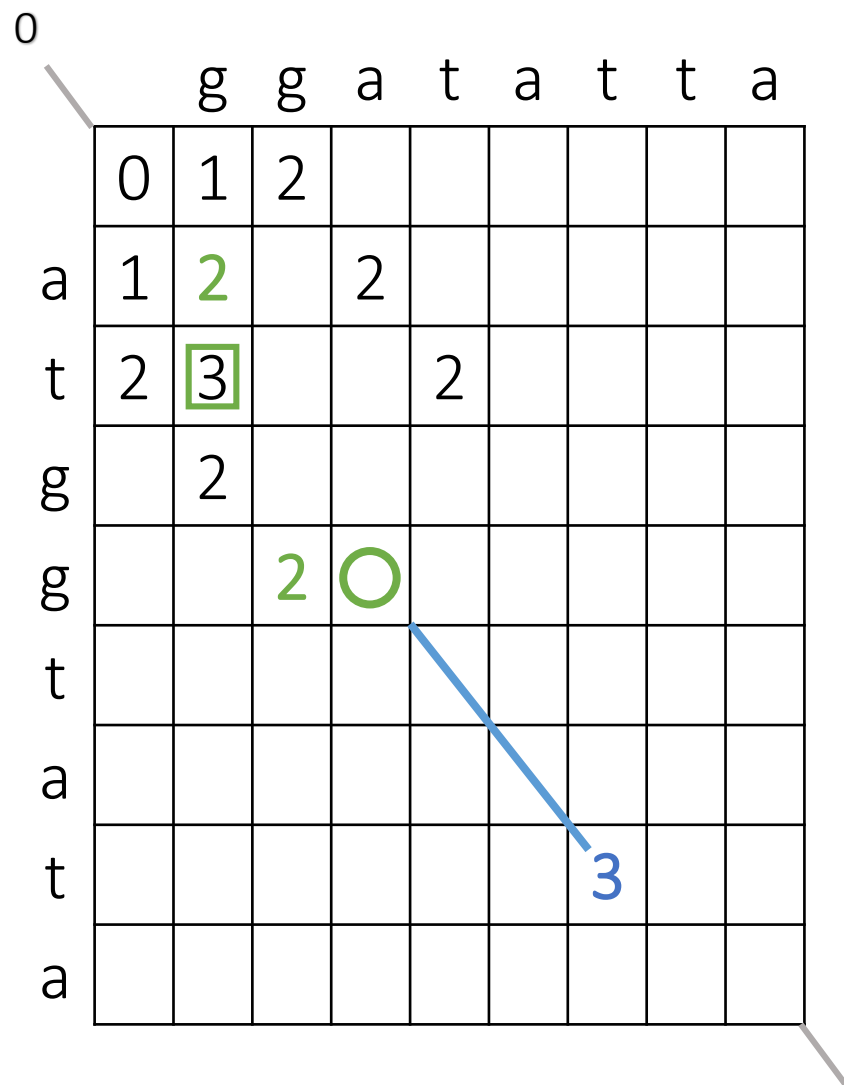
FIG. 6. Computation of $L_{new}^h(d)$ or $L_{old}^h(d)$ and update of all relevant pointers.

		g	g	a	t	a	t	t	a
	0	1	2	3	4	5	6	7	8
a	1	2	3	2	3	4	5	6	7
t	2	3	4	3	2	3	4	5	6
g	3	2	3	4	3	4	5	6	7
g	4	3	2	3	4	5	6	7	8
t	5	4	3	4	3	4	5	6	7
a	6	5	4	3	4	3	4	5	6
t	7	6	5	4	3	4	3	4	5
a	8	7	6	5	4	3	4	5	4

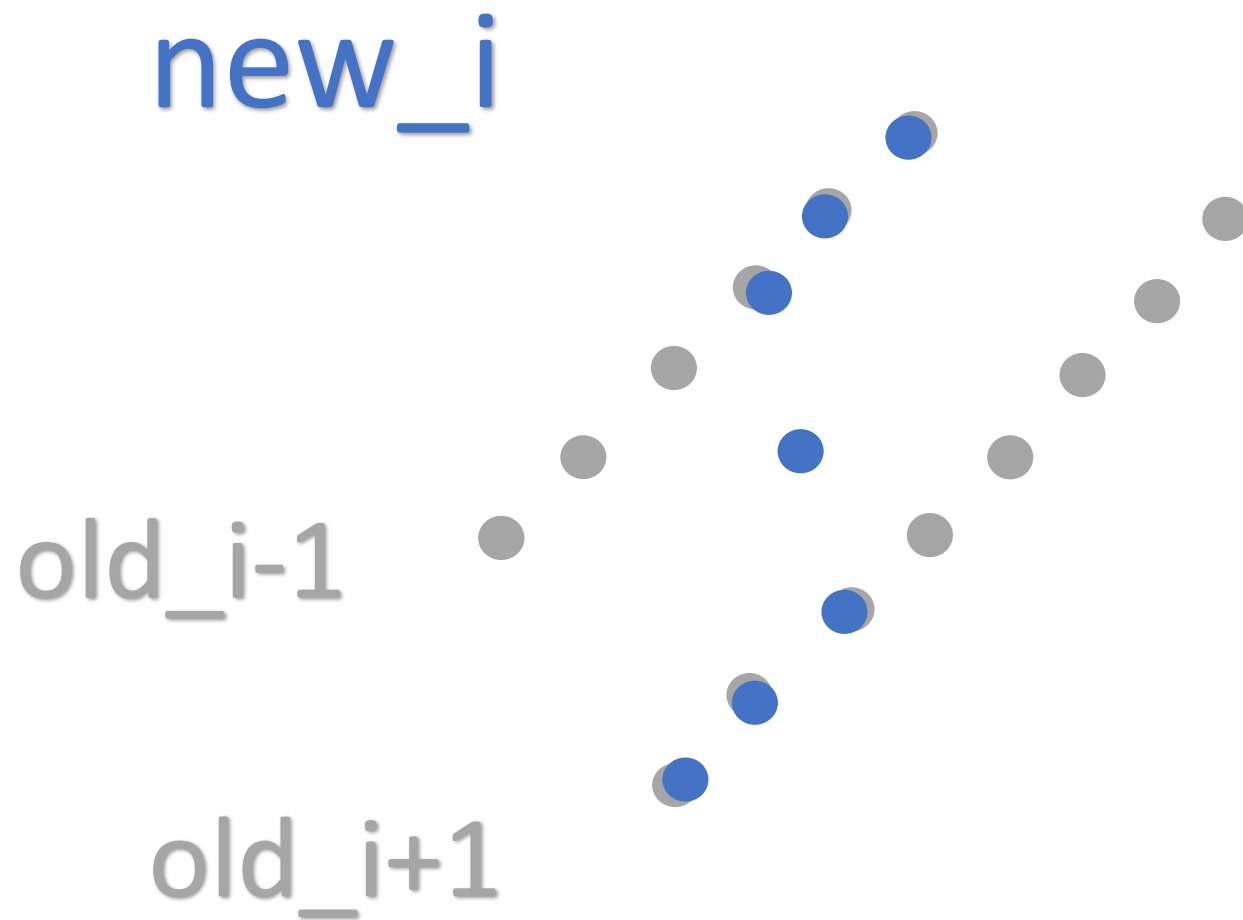
indel: 1

0	1	2	g	g	a	t	a	t	t	a
a	0	1	2	3	4	5	6	7	8	
t	1	2	3	2	3	4	5	6	7	
g	2	3	4	3	2	3	4	5	6	
g	3	2	3	4	3	4	5	6	7	
t	4	3	2	3	4	5	6	7	8	
a	5	4	3	4	3	4	5	6	7	
t	6	5	4	3	4	3	4	5	6	
a	7	6	5	4	3	4	3	4	5	
	8	7	6	5	4	3	4	5	4	

	-4	-3	-2	-1	0	1	2	3	4
0:					0				
1:				1		0			
2:			4		1		2		
3:		8		7		3		2	
4:	∞		8		8		3		2



	-4	-3	-2	-1	0	1	2	3	4
0 :					0				
1 :				1		0			
2 :			4		1		2		
3 :				7					
4 :	∞		8		8		3		2



Procedure Construct_new_LCS_wave(h, p^h, p^{h+1})

1. $L_{new}^{h+1}[-h-2 \dots p^h-1] \leftarrow L_{old}^{h+2}[-h-2 \dots p^h-1]$
2. Compute_LCS($L_{new}^{h+1}(p^h+1)$)
3. $p^{h+1} \leftarrow \begin{cases} p^h-1 & \text{if } L_{new}^{h+1}(p^h+1) < L_{old}^{h+2}(p^h+1) \\ p^h+1 & \text{otherwise} \end{cases}$
4. $L_{new}^{h+1}[p^h+3 \dots h] \leftarrow L_{old}^h[p^h+3 \dots h]$
5. Double_Link($L_{new}^{h+1}(p^h+3), L_{new}^{h+1}(p^h+1)$)

FIG. 8. Construction of L_{new}^{h+1} from L_{new}^h and auxiliary pointers.

Procedure Compute_LCS($L_x^h(d)$)

1. **if** $x = new$ **then** $\delta \leftarrow 1$ **else** $\delta \leftarrow 0$
2. $L_x^h(d) \leftarrow Slide_d \left(\max \left\{ \begin{array}{ll} L_x^{h-1}(d+1) + 1 & \text{if } d < h - \delta \\ L_x^{h-1}(d-1) & \text{if } d > -h - \delta \end{array} \right\} \right)$
3. **if** $h > 0$ **and** $d > -h - \delta$ **then**
4. $\{ \text{Double_Link}(L_x^h(d), L_x^h(d-2)) \}$
5. **if** $d < h - \delta$ **then** Double_Link($L_x^h(d), L_x^{h-1}(d+1)$)
6. $\}$

FIG. 9. Computation of $L_{new}^h(d)$ or $L_{old}^h(d)$ and update of all relevant pointers.